Revisão de Requisitos e Planejamento Scrum

PMI III — ADS31 | UNICESUSC Prof. Sérgio Murilo Schutz

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

SEMESTRE

2026-2



✅ Checklist de Requisitos Funcionais

Um bom Requisito Funcional (RF) precisa passar por este crivo antes de entrar no backlog. Use este checklist como guia prático durante a revisão do seu projeto.

1

Claro

Sem ambiguidades. Qualquer membro do time entende da mesma forma.

2

Testável

É possível escrever um caso de teste que confirme ou refute o requisito.

3

Necessário

Existe uma necessidade real do usuário ou do negócio por trás dele.

4

Rastreável

Pode ser relacionado a uma User Story, épico ou requisito de negócio.

5

Completo

Descreve o comportamento esperado do sistema sem lacunas.

6

Consistente

Não contradiz nenhum outro requisito já levantado.

🎯 Critérios INVEST para User Stories

O acrônimo **INVEST** orienta a escrita de User Stories de qualidade. Aplique-o durante a revisão para garantir que cada história seja realmente "pronta para desenvolver".



I — Independente

Pode ser desenvolvida e entregue sem depender de outra story.



N — Negociável

Não é um contrato; detalhes podem ser ajustados com o PO.



V — Valiosa

Entrega valor real e perceptível para o usuário ou negócio.



E — Estimável

O time consegue estimar o esforço com razoável precisão.



S — Small (Pequena)

Cabe dentro de uma Sprint; se grande demais, é um épico.



T — Testável

Possui critérios de aceitação claros para validação.

📐 RNF Mensuráveis com ISO 25010

Requisitos Não Funcionais (RNF) vagos como *"o sistema deve ser rápido"* são inaceitáveis. A norma **ISO 25010** fornece as categorias de qualidade e orienta como torná-los mensuráveis e verificáveis.



Desempenho

Vago: "O sistema deve ser rápido."

Mensurável: "O login deve responder em ≤ 2s para 95% das requisições sob carga de 100 usuários simultâneos."



Segurança

Vago: "O sistema deve ser seguro."

Mensurável: "Senhas armazenadas com bcrypt (custo ≥ 12); sessões expiram em 30 min de inatividade."



Confiabilidade

Vago: "O sistema deve estar sempre disponível."

Mensurável: "Disponibilidade ≥ 99,5% ao mês; recuperação de falhas em ≤ 4h (MTTR)."



Usabilidade

Vago: "O sistema deve ser fácil de usar."

Mensurável: "Usuário novato conclui cadastro em ≤ 3 min sem auxílio externo."

🔴 vs 🟢 Exemplo de RNF: Ruim × Bom

Veja na prática como transformar um RNF ruim em um RNF de qualidade, usando o contexto de um sistema de login em **React + Node.js**.

❌ RNF Ruim

"O sistema deve ser seguro e performático no login."

- Subjetivo e não verificável
- Sem métrica, sem critério de aceite
- Impossível de testar objetivamente
- Gera conflito entre time e cliente
- Não rastreável à ISO 25010

✅ RNF Bom

"O endpoint `POST /auth/login` (Node.js) deve retornar resposta em ≤ 1,5s para P95, com senha hasheada via bcrypt (custo 12) e token JWT expirado em 8h."

- Mensurável e verificável com testes
- Rastreado à categoria Desempenho/Segurança ISO 25010
- Critério de aceite claro para o time
- Testável via Jest + Artillery

🏛️ Cenários ATAM

O método **ATAM (Architecture Tradeoff Analysis Method)** avalia se a arquitetura do sistema suporta os RNFs definidos. Cada cenário descreve uma situação real que o sistema precisa enfrentar.

1. Estímulo

O que acontece? Ex.: 200 usuários tentam fazer login simultaneamente.

2. Fonte

De onde vem? Ex.: Usuários externos via navegador.

3. Ambiente

Contexto? Ex.: Sistema em produção, hora de pico.

4. Resposta

O que o sistema faz? Ex.: Processa requisições sem degradação.

5. Medida

Como avaliar? Ex.: Tempo de resposta ≤ 2s para 95% das requisições.

💡 **Dica prática:** No seu projeto, escreva pelo menos 2 cenários ATAM — um de desempenho e um de segurança — relacionados ao seu RNF mais crítico.

📖 User Stories + Gherkin (BDD)

User Stories descrevem **o quê** o usuário precisa. O **Gherkin** (BDD) detalha **como** validar esse comportamento com cenários de aceite em linguagem natural.

Formato da User Story

Como [tipo de usuário],

Quero [ação/funcionalidade],

Para que [benefício/objetivo].

Exemplo — Login

Como aluno da plataforma,

Quero fazer login com e-mail e senha,

Para que eu acesse meu painel de forma segura.

Cenário Gherkin

```
Feature: Login do Aluno

Scenario: Login com credenciais válidas
  Given que o aluno acessa /login
  And preenche e-mail "ana@ads.com"
  And preenche senha "Senha@123"
  When clica em "Entrar"
  Then é redirecionado ao dashboard
  And vê a mensagem "Bem-vinda, Ana!"

Scenario: Login com senha incorreta
  Given que o aluno acessa /login
  When insere senha errada 3 vezes
  Then conta é bloqueada por 15 minutos
```

🖥️ Validação de Protótipo

A validação com protótipos permite confirmar se os requisitos foram entendidos corretamente **antes** de iniciar o desenvolvimento, economizando tempo e evitando retrabalho.

01

Construir o Protótipo

Crie wireframes no Figma ou protótipo navegável com as telas principais (login, CRUD de produtos, dashboard).

02

Definir Roteiro de Teste

Liste tarefas que o usuário deve realizar: "Cadastre um produto", "Faça login", "Edite um registro".

03

Conduzir a Sessão

Observe sem interferir. Anote dificuldades, erros e comentários espontâneos do usuário.

04

Registrar Achados

Documente cada problema encontrado com severidade (crítico / moderado / cosmético).

05

Atualizar Requisitos

Revise as User Stories e RNFs com base nos feedbacks antes de fechar o backlog.

📦 Entregável 1 — Revisão de Requisitos

Este é o conjunto de artefatos que deve ser entregue na etapa de **Revisão de Requisitos**. Cada item será avaliado conforme a rubrica do slide seguinte.

Documento de RF

Lista de Requisitos Funcionais revisados e aprovados pelo checklist (claro, testável, necessário, rastreável, completo, consistente).

RNFs Mensuráveis

Mínimo de 3 RNFs com métricas explícitas, categorias ISO 25010 e cenários ATAM correspondentes.

User Stories + Gherkin

Mínimo de 5 User Stories no formato padrão, cada uma com pelo menos 2 cenários Gherkin (cenário feliz + alternativo).

Protótipo Validado

Link do Figma com protótipo navegável e relatório de validação (achados, severidade e ajustes realizados).

📅 **Prazo:** Entrega no AVA até a data definida pelo Prof. Sérgio Murilo. Apresentação oral em banca simulada (Slide 18).

🏆 Rubrica — Requisitos Funcionais e Não Funcionais

A avaliação de requisitos considera quatro dimensões. Veja os pesos e critérios para cada nível de desempenho.

Critério	Peso	Excelente (100%)	Satisfatório (70%)	Insuficiente (<50%)
Clareza e completude dos RFs	25%	Todos passam no checklist INVEST	Maioria aprovada, poucos ajustes	Vários RFs vagos ou incompletos
RNFs mensuráveis (ISO 25010)	30%	≥3 RNFs com métricas + ATAM	RNFs com métricas, sem ATAM	RNFs vagos, sem métricas
User Stories + Gherkin	30%	≥5 stories INVEST + cenários BDD	Stories corretas, Gherkin básico	Stories incompletas ou ausentes
Validação de protótipo	15%	Relatório com achados e ajustes	Teste realizado, sem relatório	Protótipo não validado

⚖️ **Nota:** Peso total = 100%. A média ponderada dos critérios compõe a nota do entregável de Revisão de Requisitos.

📋 Product Backlog — De RF para User Story

O **Product Backlog** é a lista ordenada de tudo que o produto precisa ter. Transformar Requisitos Funcionais em User Stories é o primeiro passo para construí-lo.

1

RF Bruto

RF01 — O sistema deve permitir que o usuário faça login com e-mail e senha.

2

User Story

Como aluno, **quero** fazer login, **para que** eu acesse o sistema com segurança.

3

Critérios de Aceite

Gherkin: cenário feliz (login válido) + alternativo (senha errada, bloqueio após 3 tentativas).

4

Item de Backlog

Story estimada em pontos, com DoR (Definition of Ready) atendida e prioridade definida.

Repita o processo para cada RF do projeto. O backlog cresce a cada refinamento e deve refletir a visão atual do produto.

🗂️ Priorização: MoSCoW + Valor vs. Esforço

Nem tudo pode ser feito na primeira Sprint. Use a técnica **MoSCoW** para classificar e a matriz **Valor × Esforço** para priorizar o que entra primeiro.

Técnica MoSCoW

- **Must Have:** Indispensável para o MVP. Ex.: Login, CRUD de produtos.
- **Should Have:** Importante, mas não crítico. Ex.: Filtros de busca.
- **Could Have:** Desejável se houver tempo. Ex.: Exportar relatório PDF.
- **Won't Have:** Fora do escopo atual. Ex.: App mobile nativo.

Matriz Valor × Esforço

Plote cada US em um quadrante:

- 🟢 **Alto valor / Baixo esforço** → Faça primeiro!
- 🟡 **Alto valor / Alto esforço** → Planeje com cuidado.
- 🟠 **Baixo valor / Baixo esforço** → Faça se sobrar tempo.
- 🔴 **Baixo valor / Alto esforço** → Evite ou descarte.



💡 Combine as duas técnicas para montar um backlog sólido e defensável na banca.

🃏 Estimativa: Fibonacci + Planning Poker

A estimativa em pontos de história usa a escala de **Fibonacci** (1, 2, 3, 5, 8, 13, 21) para capturar incerteza. O **Planning Poker** é a cerimônia onde o time estima colaborativamente.

1 ponto

Tarefa trivial. Ex.: Corrigir um typo no label do botão de login.

3 pontos

Tarefa simples. Ex.: Adicionar validação de e-mail no formulário React.

5 pontos

Tarefa média. Ex.: Implementar CRUD completo de produtos (React + Node + MongoDB).

8 pontos

Tarefa complexa. Ex.: Autenticação JWT com refresh token e controle de sessão.

13+ pontos

Épico! Deve ser quebrado em stories menores antes de entrar na Sprint.

Durante o Planning Poker, cada membro vota simultaneamente. Se houver divergência grande (ex.: 2 vs 8), os extremos justificam sua visão e o time re-vota até convergir.

🚀 Sprint Planning

O **Sprint Planning** abre cada Sprint. O time seleciona itens do backlog, define a *Sprint Goal* e decompõe as stories em tarefas técnicas no Sprint Backlog.

Parte 1 — O Quê? (~2h)

PO apresenta as stories priorizadas. Time discute escopo, critérios de aceite e tira dúvidas. Selecionam-se as stories que cabem na capacidade da Sprint (velocidade × duração).

Parte 2 — Como? (~2h)

Time decompõe cada story em tarefas técnicas. Ex.: "Implementar login" → criar rota Node.js, criar componente React, integrar JWT, escrever testes Jest.

Sprint Goal

Uma frase que sintetiza o objetivo da Sprint. Ex.: *"Ao final desta Sprint, o usuário consegue se cadastrar e fazer login no sistema."*

Capacidade do Time

Calcule: (membros × horas/dia × dias da Sprint) − folgas e reuniões. Não comprometa mais do que 80% da capacidade.

📜 Artefatos Scrum e Definition of Done

O Scrum define três artefatos principais e a **Definition of Done (DoD)**, que é o contrato de qualidade que toda story deve cumprir para ser considerada "pronta".



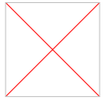
Product Backlog

Lista viva de todas as funcionalidades desejadas, ordenada por valor e mantida pelo PO.



Sprint Backlog

Subconjunto selecionado para a Sprint atual, com tarefas técnicas e responsáveis.



Incremento

Software funcionando, integrado e potencialmente entregável ao final de cada Sprint.

✅ Definition of Done (DoD)

- Código revisado (code review aprovado)
- Testes unitários escritos (cobertura ≥ 70%)
- Critérios de aceite Gherkin passando
- Sem warnings críticos no linter
- Deploy em ambiente de homologação
- Documentação atualizada no README

📦 Entregável 2 — Planejamento Scrum

O segundo entregável documenta o planejamento ágil do projeto. Todos os artefatos devem estar no repositório do grupo no AVA/GitHub até a data estipulada.

Product Backlog

Mínimo de 10 User Stories priorizadas via MoSCoW, com estimativas em pontos de história (Fibonacci) e critérios de aceite.

Sprint Backlog (Sprint 1)

Stories selecionadas para a Sprint 1, decompostas em tarefas técnicas com responsáveis e estimativas em horas.

Sprint Goal

Objetivo claro e mensurável para a Sprint 1, aprovado por todo o time e alinhado ao MVP do projeto.

Definition of Done

DoD documentada e acordada pelo time, com critérios técnicos específicos do stack React + Node.js utilizado.

Quadro Kanban

Board configurado (GitHub Projects, Trello ou Jira) com colunas: To Do / In Progress / In Review / Done.

🏆 Rubrica — Planejamento Scrum

A avaliação do planejamento Scrum considera cinco critérios com pesos distintos. Use esta rubrica como guia de autoavaliação antes de entregar.

Critério	Peso	Excelente (100%)	Satisfatório (70%)	Insuficiente (<50%)
Product Backlog (priorização MoSCoW)	25%	≥10 stories, MoSCoW + Valor×Esforço	Stories criadas, priorização básica	Backlog incompleto ou sem prioridade
Estimativas Fibonacci / Poker	20%	Todas stories estimadas, justificativa	Maioria estimada, sem justificativa	Estimativas ausentes ou arbitrárias
Sprint Planning documentado	25%	Goal + backlog + capacidade calculada	Goal e backlog, sem capacidade	Planning não documentado
Definition of Done	15%	DoD técnica específica do stack	DoD genérica e acordada	DoD ausente ou genérica demais
Quadro Kanban funcional	15%	Board atualizado com todas as tarefas	Board criado, parcialmente atualizado	Board ausente ou vazio

📊 **Nota final:** Média ponderada dos dois entregáveis (Requisitos + Scrum) compõe a nota do PMI III.